A D PATEL INSTITUTE OF TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

RESTAURANT BILLING SYSTEM

A Mini Project Report

A report submitted
In partial fulfilment of the requirements for the Degree of
Bachelor of Technology
in
Information Technology
Semester – V

Subject:
Programming with Python (202044504)

By

Het Patel (12402080601045)

Department of Information Technology
A D Patel Institute of Technology
New Vallabh Vidyanagar, Vitthal Udyognagar – 388121
Anand (Gujarat), India

A D PATEL INSTITUTE OF TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the mini project work embodied in this report entitled **"Restaurant Billing System"** was carried out by **Het Patel (12402080601045)** at A D Patel Institute of Technology for partial fulfilment of the requirements of the Bachelor of Technology degree awarded by Charutar Vidya Mandal (CVM) University. This project work has been carried out under the guidance and supervision of the faculty guide and is to the satisfaction of the department.

Date: _____
Place: New Vallabh Vidyanagar

Lab Teacher	Head of Department
Nayan Mali Assistant Professor	Dr. Priyang Bhatt Associate Professor & Head

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to A D Patel Institute of Technology and the Department of Information Technology for providing the opportunity and resources to undertake this mini project, “Restaurant Billing System.”

I am thankful to my faculty guide and lab teacher for their guidance, suggestions, and support throughout the development of this project. Their feedback helped me understand Python programming, graphical user interface development, data structures, calculations, and file handling.

I also thank my classmates and everyone who provided assistance or encouragement during the project. Finally, I am grateful to my family for their continuous support.

Het Patel
(12402080601045)

TABLE OF CONTENTS

Section	Topic
Abstract	Project summary
1	Introduction and Problem Statement
2	Project Objectives and Scope
3	Functional and Technical Requirements
4	System Architecture and Folder Structure
5	Python Concepts and Technologies Used
6	Menu and Order Management
7	Billing, Tax and File Handling
8	Graphical User Interface Design
9	Validation and Error Handling
10	Testing and Test Cases
11	Setup and Run Instructions
12	Limitations and Future Improvements
13	Conclusion
References	Documentation sources

List of Tables

Table 1: Functional requirements
Table 2: Technical requirements
Table 3: Application layers and responsibilities
Table 4: Restaurant menu fields
Table 5: Bill calculation example
Table 6: Input controls and buttons
Table 7: Validation checks and handling
Table 8: Manual test cases
Table 9: Setup commands

List of Figures

Figure 1: Architecture and data flow of the application
Figure 2: Main window layout of the Restaurant Billing System

ABSTRACT

Restaurant Billing System is a desktop application concept developed using Python and Tkinter to help restaurant staff prepare customer bills quickly and accurately. The application allows a user to select menu items, enter quantities, add items to an order, calculate the subtotal and applicable tax, and display the final payable amount. It can also save bill details locally for later reference.

The project demonstrates important Python concepts including lists, dictionaries, functions, classes, exception handling, GUI programming, and file handling. The system is designed as a beginner-level mini project that combines core programming concepts with a practical restaurant workflow.

Keywords: Python, Tkinter, Restaurant Billing System, Menu, Order Management, Bill Calculation, JSON, GUI.

1. Introduction and Problem Statement

1.1 Background

Restaurants handle multiple menu items and customer orders every day. Preparing bills manually can take time and may lead to calculation errors, especially when several items and quantities are involved. A computer-based billing application provides a structured way to select items, calculate amounts, and present a clear bill.

1.2 Problem Statement

The problem addressed by this project is to design and develop a user-friendly desktop application that allows restaurant staff to select menu items, record quantities, calculate the total bill, and save bill information using Python.

1.3 Motivation

This project applies programming concepts to a real-world business process. It demonstrates how a graphical interface, data structures, validation, and automatic calculations can improve speed and consistency in routine billing.

2. Project Objectives and Scope

2.1 Objectives

• Create a simple graphical interface for restaurant billing.
• Maintain a menu containing item names and prices.
• Allow the user to select items and enter quantities.
• Calculate item totals, subtotal, tax, and final amount.
• Display order details in a bill table.
• Validate quantities and prevent invalid entries.
• Save bill records locally for future reference.

2.2 Scope

In scope: displaying a predefined menu, adding items to an order, changing quantities by adding line items, calculating totals, generating a bill view, clearing an order, and saving bill records locally. The project is intended for a small restaurant or academic demonstration.

Out of scope: online payments, inventory synchronisation, multi-branch management, customer loyalty, cloud synchronisation, and integration with commercial point-of-sale hardware. These features may be considered for future development.

3. Functional and Technical Requirements

3.1 Functional Requirements

ID	Requirement
FR1	The user can view available menu items and prices.
FR2	The user can select a menu item and enter a quantity.
FR3	The application adds selected items to the current order.
FR4	The application calculates each line amount and subtotal.
FR5	The application calculates tax and the final payable amount based on configured rates.
FR6	The user can generate, clear, and review a bill.
FR7	Bill details can be saved to a local file.

3.2 Technical Requirements

Aspect	Technology / requirement
Programming language	Python 3
GUI library	Tkinter and ttk
Data structures	List and dictionary
Data storage	JSON text file (optional local persistence)
Operating system	Windows, macOS, or Linux with Tkinter support
Additional packages	No external packages required for the core application

3.3 Non-Functional Requirements

• Usability: users should be able to prepare a bill through the graphical interface.
• Accuracy: line totals and final totals should be calculated consistently.
• Reliability: invalid entries should be handled without terminating the application.
• Persistence: saved bill records should remain available after reopening the application.
• Portability: the application should run wherever Python 3 and Tkinter are available.

4. System Architecture and Folder Structure

4.1 Architecture Overview

The application follows a simple single-program architecture. The Tkinter interface collects item and quantity input. Application logic validates input, updates the order list, calculates line totals and taxes, and calls file-handling functions to save bill records.

Layer / Module	Responsibility
Presentation layer	Creates menu selectors, quantity entry, buttons, bill table, and totals.
Application logic	Validates input and performs add-item, remove/clear, and billing operations.
Data structure layer	Stores menu items and order lines in lists and dictionaries.
File storage layer	Reads and writes bill history in a local JSON file.

4.2 Data Flow

1. User selects a menu item and quantity.
2. Input validation checks the quantity and selected item.
3. The order line is added to the order list.
4. Line totals, subtotal, tax, and grand total are recalculated.
5. The bill display is refreshed and the record can be saved.

4.3 Folder Structure

```
Restaurant_Billing_System/\n |-- restaurant_billing.py\n |-- README.md\n |-- bills.json
```

5. Python Concepts and Technologies Used

5.1 Lists

A list stores multiple order lines. Each selected item is added to the list, allowing the program to display all ordered items and calculate the subtotal by iterating through the records.

```
order = []\norder.append({"item": "Veg Burger", "qty": 2, "price": 120.0})
```

5.2 Dictionaries

A dictionary stores related information using key-value pairs. Each menu item can contain a name and price, while each order line contains an item name, quantity, price, and line total.

```
menu_item = {"name": "Veg Burger", "price": 120.0}\norder_line = {"item": "Veg Burger", "qty": 2, "price": 120.0}
```

5.3 Tkinter

Tkinter is Python's standard GUI library. The project can use labels, entry fields, comboboxes, buttons, message boxes, and a ttk Treeview table to allow staff to operate the billing system without command-line input.

5.4 Functions and Classes

The program can be organised into functions or a RestaurantBilling class. Separate methods make it easier to validate input, add items, calculate totals, refresh the bill display, and save records.

Function / method	Purpose
add_item()	Validates the selected item and quantity, then adds an order line.
calculate_total()	Calculates subtotal, tax, and grand total.
refresh_bill()	Updates the bill table and total labels.
clear_order()	Removes current order lines and resets displayed totals.
save_bill()	Writes a completed bill record to the local JSON file.
load_menu()	Loads or defines the menu items and prices.

6. Menu and Order Management

6.1 Menu Structure

The menu is a collection of available food and beverage items. Each item has a name and a price. For a simple academic project, the menu can be predefined in the Python program; it can later be moved to a separate JSON file.

Field	Type	Description
item_id	Integer / string	Unique identifier for the menu item.
name	String	Name displayed to the restaurant staff.
price	Float	Price of one unit in rupees.
category	String	Optional group such as Starter, Main Course, Dessert, or Beverage.

6.2 Adding Items to an Order

The user chooses a menu item, enters a positive whole-number quantity, and clicks Add Item. The program verifies the input, obtains the unit price from the menu, computes the line amount, and adds the item to the order list.

6.3 Displaying Order Details

The bill table displays the item name, quantity, unit price, and line amount. The table is refreshed whenever an item is added or the order is cleared, ensuring the displayed bill matches the current order.

6.4 Bill Calculation

For each order line, the line amount is calculated as quantity multiplied by unit price. The subtotal is the sum of all line amounts. Tax is calculated using the configured rate, and the grand total is the subtotal plus tax. The tax rate should be configured according to the intended demonstration and applicable rules.

```
line_amount = quantity * unit_price\nsubtotal = sum(line['amount'] for line in order)\ntax_amount = subtotal * tax
```

6.5 Example Bill

Item	Qty	Unit price	Amount
Veg Burger	2	■120.00	■240.00
Masala Dosa	1	■90.00	■90.00
Lemon Soda	2	■40.00	■80.00
Subtotal			■410.00
Tax (example 5%)			■20.50
Grand Total			■430.50

7. Billing Records and File Handling

7.1 Purpose of File Handling

File handling allows completed bills to be stored so they can be reviewed after the application closes. JSON is suitable for a small demonstration because it is human-readable and can represent lists and dictionaries.

7.2 Saving a Bill

When the user completes an order and chooses Save Bill, the application can save a record containing a bill number, date and time, ordered items, subtotal, tax amount, and grand total. The program should handle file errors and display a helpful message.

```
with open("bills.json", "w", encoding="utf-8") as file:\n    json.dump(bills, file, indent=4)
```

7.3 Loading Records

When the application starts, the program may load existing bill records from bills.json. If the file does not exist, it can start with an empty history. Invalid JSON should be handled with exception handling rather than crashing the application.

7.4 Example JSON Record

```
{\n  "bill_no": 1001,\n  "date": "2026-10-26",\n  "items": [{"item": "Veg Burger", "qty": 2, "price": 120.0}],\n
```

8. Graphical User Interface Design

8.1 Main Window

The main window is titled “Restaurant Billing System.” It contains a menu and quantity input area, an Add Item button, an order table, subtotal/tax/grand-total labels, and actions to generate, save, or clear a bill.

8.2 Input Controls

Control	Purpose
Menu combobox	Allows the user to choose a food or beverage item.
Quantity entry	Accepts the number of units ordered.
Add Item button	Validates and adds an item to the current order.
Order table	Shows item, quantity, unit price, and amount.
Generate / Save Bill button	Displays and saves the completed bill.
Clear Order button	Clears the current order after confirmation if needed.

8.3 Bill and Summary Display

The bill table provides a clear overview of the selected items. The subtotal, tax, and grand total are recalculated after each change. A scrollbar can be used when the order contains many entries.

8.4 User Interaction Flow

Launch application → View menu → Select item and quantity → Click Add Item → Review order table → Calculate subtotal and tax → Generate bill → Save bill if required.

9. Validation and Error Handling

The application validates user input before adding an order line. These checks help prevent incomplete or incorrect bills.

Validation	Expected handling
No menu item selected	Show a warning and do not add an item.
Quantity is empty or non-numeric	Show an error message and reject the entry.
Quantity is zero or negative	Reject the quantity; only positive whole numbers are accepted.
Menu price is missing or invalid	Do not calculate the bill; show an error.
No items in current order	Warn the user before generating a bill.
JSON file is missing	Start with empty bill history.
JSON file is malformed or unreadable	Handle the exception and show a useful message.

10. Testing and Test Cases

Testing can be performed manually by entering different menu selections and quantities and checking whether the application behaves as expected.

ID	Test input / action	Expected result
T1	Add one valid menu item.	Item appears in the order table.
T2	Enter quantity 0.	Quantity is rejected.
T3	Enter alphabetic quantity.	Validation message appears.
T4	Add two different items.	Both items appear and subtotal is correct.
T5	Calculate tax and grand total.	Tax and final amount are correct for configured rate.
T6	Generate a bill with no items.	Warning appears; no empty bill is generated.
T7	Save a completed bill.	Bill record is stored in JSON.
T8	Restart after saving.	Saved history can be loaded if implemented.
T9	Clear the current order.	Order table and totals reset.

10.1 Expected Outcome

The system should allow valid items to be added, calculate item amounts and the final bill correctly, display order details, and save bill information when that feature is enabled. Invalid input should be handled through user-friendly messages rather than causing the application to terminate.

11. Setup and Run Instructions

11.1 Prerequisites

• Python 3 installed on the system.
• Tkinter available in the Python installation.
• A code editor or terminal to run the Python file.

11.2 Execution Steps

1. Create or extract the Restaurant_Billing_System project folder.
2. Open a terminal in the project folder.
3. Run the Python program.
4. Select menu items, enter quantities, and generate a bill.

```
python restaurant_billing.py\n\n# On macOS/Linux, if required:\npython3 restaurant_billing.py
```

Command	Purpose
python restaurant_billing.py	Starts the application on systems where python points to Python 3.
python3 restaurant_billing.py	Starts the application on macOS/Linux where python3 is used.

The application can create or update bills.json when bill records are saved. No external Python packages are required for the core application.

12. Limitations and Future Improvements

12.1 Current Limitations

- The basic project is designed for a single computer and a small restaurant workflow.
- The menu may be predefined and require code changes to update prices.
- It does not integrate with online payment gateways or printers by default.
- It does not provide stock management, user roles, or multi-branch reporting.
- Local JSON files are not a substitute for a secured production database.

12.2 Possible Future Improvements

- Add a database such as SQLite or MySQL for structured bill history.
- Add menu editing and category management.
- Add discount and configurable tax options with appropriate controls.
- Add printable receipts and PDF export.
- Add daily, weekly, and monthly sales reports.
- Add staff login and role-based access.
- Integrate inventory tracking and payment methods.
- Add backup and restore functionality.

13. Conclusion

The Restaurant Billing System mini project demonstrates how Python can be used to develop a practical desktop application. Tkinter provides the graphical interface, lists and dictionaries organise menu and order data, and calculation functions determine the subtotal, tax, and final payable amount.

The system supports selecting menu items, entering quantities, displaying an itemised order, calculating totals, and saving bill details when implemented. Input validation and exception handling improve reliability and make the application easier to use. The project applies core Python concepts in an integrated system and can be extended with database storage, reports, receipt printing, and inventory features.

REFERENCES

[1] Python Software Foundation, “Python 3 Documentation.” https://docs.python.org/3/

[2] Python Software Foundation, “Tkinter — Python interface to Tcl/Tk.” https://docs.python.org/3/library/tkinter.html

[3] Python Software Foundation, “json — JSON encoder and decoder.” https://docs.python.org/3/library/json.html

[4] Python Software Foundation, “Built-in Types — Lists and Dictionaries.” https://docs.python.org/3/library/stdtypes.html

[5] Python Software Foundation, “Built-in Functions — open().” https://docs.python.org/3/library/functions.html#open

Project note

This report is structured as an academic mini-project report. The implementation details describe a suitable design for a Restaurant Billing System; the exact functions available depend on the final Python source code created for the project.

Het Patel | 12402080601045 | IT A